Lists

Coming soon!

Control Flow

1. Write an if statement that takes a single integer `n` and prints "even" if `n` is divisible by 2, "odd" otherwise. Test your code with `n <- 7` and `n <- 16`.

2. Write an if statment that takes an exam score like `score <- 93` and returns `"A"` if the score is greater than 90.

3. Extend your previous answer to return letters grades of A, B, C, or "no pass" if the score is in the 90s, 80s, 70s, or less than 70, respectively.

4. On the lines provided below, complete the code to iterate through the vector `x` and print its contents. For this question, the iterator, `i`, should take as its values the character string elements of `x`. (*Hint: see Option 3 in the slides*)

```
x <- c("apple", "banana", "cherry")
```

```
for (i in _______) {

    _________(________)

}
```

5. Repeat the question but this time, the iterator, `i`, should take as its values the integers from 1 to 3. (*Hint: see Option 1 in the slides*)

```
for (i in _______) {

    _________(________)

}
```

6. Write a for loop that uses `x` and prints the following output:

```
Fruit 1: apple
Fruit 2: banana
Fruit 3: cherry
```

7. Using a for-loop (don't cheat with vectorization!), change each of the temperatures of `temps_c` one-by-one so that they are converted from degrees Celcius to Fahrenheit.

```
temps_c <- c(0, 10, 20, 30)
```

8. For-loops, like if-else statments, can be *nested*: inside the two for-loops below, your code has access to both of the iterators `i` and `j`. Fill in the code below to turn `m` from a matrix of 0s to a multiplication table showing the products of the integers 1, 2, 3 with one another.

```
m <- matrix(rep(0, 9), nrow = 3)

for (i in __________) {

    for (j in __________) {

        m[__________] <- __________
    }
}
```

Run the code in R to check your answer. As a cherry on top, make the row and column names of this multiplication table easier to read by reassigning their names with `rownames(m) <- ____` and `colnames(m) <- ____`.

9. *For fun*: Run the following code and view the result.

```
plot(0, 0)

for(i in 1:100) {
  points(runif(1, min=0, max=1), runif(1, min=0, max=1))
}
```

`plot(0, 0)` makes a scatterplot with a single point at (0, 0). `points(x, y)` will add a point to your plot at (x, y) and `runif()` works like `rnorm()` except it generates 1 random uniform number between 0 and 1 instead of a random normal number (from a bell curve).

Modify this code so that your random points appear uniformly along the unit circle (a circle centered at (0, 0) with radius one) instead of uniformly in the upper right quadrant.